1. Methodology and motivation

З EDA я виділила кілька важливих моментів. Пропуски становлять 2-3% майже в кожній колонці, а середній score з пропусками й без однаковий, тому можна сказати, що пропуски незалежні. Без пропусків залишається лише 302 244 рядки (втрата ~31%), тож краще імпутувати. Категорії program_track, rest_quality та learning_routine мали дублі через регістр і пробіли. Також були неможливі значення: study_time до 32 годин на добу та attendance_rate до 150% (747 рядків > 100%). study_time і study_minutes виявилися однією величиною в різних одиницях (corr 0.95). За кореляцією найсильніше з target пов'язані study_minutes (0.76), engagement_index (0.74) і study_time (0.73). За середнім score помітно відрізняються групи rest_quality, learning_routine і campus_resources, тоді як для registration_group, assessment_level та home_internet середній score майже однаковий у всіх групах, тож сигналу в них немає.

У розділі Feature Engineering, спираючись на EDA, ознаки без сигналу (registration_group, assessment_level, home_internet, intake_marker) видалила. Для найсильніших сигналів додала взаємодії: study_minutes × attendance_rate, study_time × attendance_rate і study_minutes / sleep_duration. Для категорій, що найбільше впливають на score (rest_quality, learning_routine, campus_resources), додала прапорці good_conditions і bad_conditions. Оскільки study_minutes і study_time майже дублюють одна одну, спробувала заповнити пропуски в study_minutes значеннями study_time × 60.

Для очищення я привела категорії до єдиного вигляду. Неможливі значення (study_time > 24 і attendance_rate > 100) замінювала на NaN. Поріг 24 це фізична межа доби, а суворіший поріг 9 пробувала лише в експерименті зі study_minutes. Пропуски в числових ознаках заповнюю медіаною, у категоріальних ознаках пропуски отримують окрему категорію "Missing". Кодування залежить від моделі: OneHot для лінійної регресії та XGBoost, Ordinal для дерева й лісу (швидше й не збільшує кількість колонок), нативні категорії для CatBoost. Вся імпутація й кодування лежать у Pipeline / ColumnTransformer, тому статистики рахуються тільки на train-фолді й витоку немає. До test застосовується та сама очистка.

Для тренування та валідації я використала 5-fold KFold (shuffle=True, random_state=42) і однакові фолди для всіх експериментів, щоб результати були порівнювані. Для оцінки моделей використовувала середній val RMSE по фолдах та OOF RMSE, а також train RMSE для контролю перенавчання. Для бустингу рання зупинка йде по окремих 10% train-частини фолда з early_stopping_rounds = 50 і максимумом 3000 дерев. Для фінальної моделі беру середню найкращу кількість раундів, помножену на 1.1.

Моделі порівнювались в порядку зростання складності: baseline (середнє), лінійна регресія, DesicionTree, Random Forest, XGBoost, XGBoost+feature engineering, CatBoost, CatBoost+feature engineering, stacking: RandomForest, XGBoost і CatBoost (метамодель Ridge). Для стекінгу використала nested CV: у кожному з 5 зовнішніх фолдів train-частину ще раз ділила на 3 внутрішні фолди, на яких RF, XGBoost і CatBoost робили out-of-fold прогнози, а на них навчала метамодель Ridge. Потім базові моделі перенавчала на всій train-частині фолда, а метамодель оцінювала на зовнішньому валідаційному фолді, якого не бачила жодна модель, тому оцінка без витоку.

Гіперпараметри я підбирала по одному за раз, фіксуючи найкращі попередні значення, а не повним перебором для збереження часу. Мінус такого підходу в тому, що

можна пропустити взаємодії між параметрами. Для дерева підбирала max_depth, потім min_samples_leaf. Для лісу - n_estimators, max_depth і max_features. Для XGBoost - GridSearchCV з lr = 0.01, max_depth ∈ {3, 4} і n_estimators ∈ {4000, 5000, 6000}. Для CatBoost - learning_rate та l2_leaf_reg.

2. Results and analysis

Model	**Val RMSE**	**Test RMSE**
1. Константа	18.9177	18.89815
2. Linear Regression	9.1678	9.1788
3. Desicion Tree	13.412	-
4. Tuned Desicion Tree	9.5036	-
5. RandomForest	9.2573	-
6. Tuned RandomForest	9.1401	9.14881
7. XGBoost	9.0037	9.0063
8. Tuned XGBoost	8.9397	8.94205
9. Tuned XGBoost without weak features	8.9355	-
10. Tuned XGBoost with study_minute_final	8.9797	-
11. Tuned XGBoost with feature interactions	8.9298	-
12. Tuned XGBoost with categorical interactions	8.9293	-
13. Final XGBoost	8.924	8.9354
14. RandomForest with feature engineering	9.1136	-
15. CatBoost	8.9227	-
16. CatBoost with feature engineering	8.9186	-
17. Final CatBoost	8.9165	8.9145
18. Stacking	8.9105	8.9161

1-2. Baseline і лінійна регресія. Константа не використовує ознаки, тому це точка відліку. Лінійна регресія одразу дала великий стрибок, бо target майже лінійно залежить від study_minutes, engagement_index і study_time (кореляції 0.73–0.76).
3-4. Decision Tree. Дерево без обмежень перенавчилось (train RMSE = 0, val 13.14). Обмеження глибини і min_samples_leaf зменшили перенавчання і покращили результат до 9.50. Гірше за лінійну модель.

5-6. Random Forest. Усереднення дерев зменшує дисперсію, тому ліс кращий за одне дерево. Збільшення кількості дерев дає мало користі, а max_depth=13 і max_features=0.5 дають невелике покращення. Результат кращий за DesicionTree та лінійну регресію.
7-8. XGBoost. Бустинг будує неглибокі дерева послідовно, і кожне виправляє помилки попередніх. Тому він краще за ліс ловить і лінійну залежність, і дрібні взаємодії. Тюнінг із ранньою зупинкою дав ще одне покращення.
9. Видалення слабких ознак. Ці ознаки мали важливість ≈ 0.0005, тобто шум. Метрика не погіршилась, модель стала простішою, тож залишила.
10. Відновлення study_minutes. Кореляція між study_time і study_minutes 0.95, а не 1, тому відновлені значення неточні. Колонка стала сумішшю значень різної точності, а нової інформації не додалось (+0.044). Відкинула.
11. Взаємодії числових ознак. Добуток study × attendance відображає «ефективне навчання», і ці ознаки несуть основний сигнал. Невелике, але стабільне покращення.
12. Взаємодії категоріальних ознак. Ефекту майже немає, бо групи нечисленні; імовірно дерева вже будують такі комбінації.
13. Final XGBoost. lr=0.01 з більшою кількістю дерев дав лише тисячні частки на CV
14. Нові ознаки для RF. Невелике покращення, але RF все одно слабший за бустинг, бо гірше використовує нові ознаки.
15-17. CatBoost. Нативна обробка категорій дали кращий результат, ніж XGBoost з тими ж lr і depth. Нові ознаки теж дали покращення.
Після тюнінгу lr=0.03, l2_leaf_reg=3 - найкращий результат серед одиночних моделей.
18. Stacking. Nested CV найкращий (8.9105), але на Kaggle стекінг гірший за CatBoost (8.916 проти 8.9145). Базові моделі сильно скорельовані, тому комбінація майже не додає нової інформації, а різниця лежить у межах шуму.

3. Final model and run instructions

Фінальною моделлю обрано CatBoostRegressor (learning_rate = 0.03, depth = 6, l2_leaf_reg = 3), навчений на всіх даних train з набором ознак, до якого входять очищені вихідні ознаки без чотирьох слабких (intake_marker, registration_group, assessment_level, home_internet), три числові взаємодії (weighted_study, weighted_time, study_efficiency) та два прапорці (good_conditions, bad_conditions). Модель має Val RMSE 8.9165 і Test RMSE 8.91449.

Код виконується в Google Colab з GPU; потрібні бібліотеки: pandas, numpy, scikit-learn, catboost. Відтворення прогнозів складається з чотирьох послідовних етапів ноутбука: завантаження даних із Kaggle; очищення текстових категорій (strip, lower) у train і test; побудова ознак (видалення слабких ознак, числові та категоріальні взаємодії), що дає набір X_flags; навчання CatBoost на всьому train і прогноз для test. Етапи виконуються в порядку їх розташування в ноутбуці. У всіх моделях і розбиттях використано seed 42.